Day 139: Week 20 Build: A Conditional DCGAN, Scored, and the Forward Diffusion Process

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 7 | Complete study guide

Watch: The diffusion model (DDPM) – the basics | Generative AI (TileStats) | What are Diffusion Models? (Ari Seff)

Script: `day139_gan_build_and_forward_diffusion.py` · every number in this guide is copied by program from `day139_run_output.txt`, the real output of that script.

The week's build. The 43 functions and classes of Days 133–138's exercise files are pasted in verbatim, checked character by character, and assembled into one model: a conditional DCGAN trained on all 60,000 MNIST training images and scored with Day 138's metrics. After 10 epochs it reaches FID 20.200 and draws the asked-for digit 0.834 of the time. Day 133's Gaussian class models, fitted in seconds, score FID 41.405 at best but are more often named as the digit asked for (0.966). The day then starts Week 21's subject: the forward diffusion process, which turns data into pure noise one small step at a time, measured on Day 135's ring of eight Gaussians and on MNIST digits.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

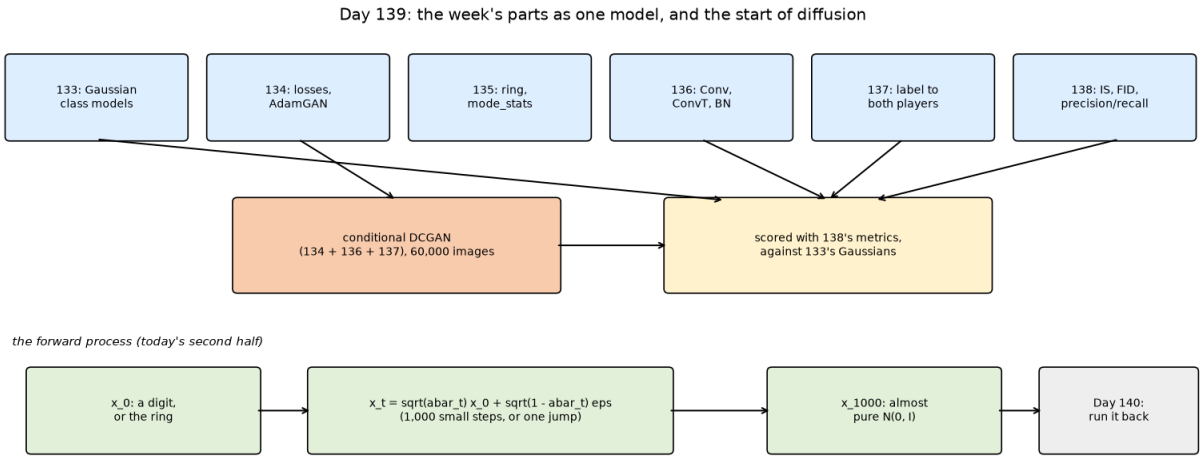
A **build day** puts the week's parts together. Week 20 built generative models: Gaussian class models (Day 133), the GAN and its losses (134), mode collapse measured on a ring of Gaussians (135), convolutional players (136), the label given to both players (137) and the scores (138).

The **forward diffusion process** is the first half of a diffusion model. It adds a little Gaussian noise at each of T = 1,000 steps, shrinking the signal just enough that the total variance stays the same. After the last step nothing of the data is left: x_T is pure noise. A diffusion model learns to run this backwards (Day 140).

REAL-WORLD SCENARIO

A photograph left in the sun fades a little each day until nothing is left. Fading is easy and needs no skill; the forward process is that fading, written down exactly. Restoring the photograph is the hard direction, and it is what the model will learn.

SYSTEM DIAGRAM



The week's parts in one model, and the forward process that starts Week 21.

Data: MNIST (70,000 handwritten digits, cached by Day 126). Today has no exercise: it is the build day, and it uses the six exercise files of Days 133–138. Day 116's softmax regression is imported unchanged.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Checks: the parts are verbatim, and the forward process's shortcut

DEFINITION — start here, no prior knowledge assumed

The script reads its own source and each exercise file, and confirms that every function and class appears character for character. Then the forward process is checked two ways on 200,000 numbers, half at –1 and half at +1: by taking all 1,000 small steps, and by jumping straight to step t with one formula.

REAL-WORLD SCENARIO

Checking that a recipe's shortcut ('bake 40 minutes at 180°') gives the same cake as following every minute of the long version.

MATHEMATICS

$$\text{one step: } x_t = \sqrt{(1-\beta_t)}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon, \quad \varepsilon \sim N(0,1)$$

$$A_t = (1-\beta_1)(1-\beta_2)\ldots(1-\beta_t) \;\Rightarrow\; x_t = \sqrt{A_t}\, x_0 + \sqrt{(1-A_t)}\, \varepsilon \quad \text{(one jump)}$$

$$\text{if Var } x_0 = 1: \quad \text{Var } x_t = A_t + (1-A_t) = 1, \quad E[x_t\, x_0] = \sqrt{A_t}$$

IMPLEMENTATION

```
def schedule(kind="linear", T=1000):
    if kind == "linear":
        betas = np.linspace(1e-4, 0.02, T)
    else:
        f = lambda t: np.cos((t / T + 0.008) / 1.008 * np.pi / 2) ** 2
        abar = f(np.arange(T + 1)) / f(0); betas = np.clip(1 - abar[1:] / abar[:-1], 0, 0.999)
    abar = np.concatenate([[1.0], np.cumprod(1 - betas)])
    return betas, abar
def q_step(x, beta, rng):
    return np.sqrt(1 - beta) * x + np.sqrt(beta) * rng.standard_normal(x.shape)
def q_sample(x0, t, abar, rng):
    return np.sqrt(abar[t]) * x0 + np.sqrt(1 - abar[t]) * rng.standard_normal(x0.shape)
```

ACTUAL OUTPUT

```
functions and classes from day133_exercise.py to day138_exercise.py found here, character for character: 43 of
    43
linear schedule: beta from 0.0001 to 0.0200 over 1000 steps; alpha-bar_1000 = 4.04e-05, so sqrt(alpha-bar) =
    0.0064 of the signal is left
     t   1,000-step route: mean, variance, E[x x_0]    shortcut: mean, variance, E[x x_0]  formula E[x x_0] =
         sqrt(abar)
    10         0.0038    1.0001       0.9991               0.0037    1.0000     0.9991             0.9991
   100         0.0043    1.0002       0.9474               0.0029    1.0014     0.9481             0.9471
   300         0.0016    1.0004       0.6294               0.0015    0.9932     0.6259             0.6296
  1000         0.0002    0.9969       0.0069              -0.0004    1.0011     0.0076             0.0064
the data has variance 1, and so does every x_t: each step takes away exactly as much signal variance as it
    adds noise
Actual output from this lesson's run.
```

What it means

All 43 of 43 parts are present verbatim. With the linear schedule only 0.0064 of the signal is left after 1,000 steps. The two routes agree: at t = 300 the remaining signal is 0.6294 by the long route, 0.6259 by the shortcut and 0.6296 by the formula, and the variance stays near 1 at every t (0.9969 to 1.0014). The differences are sampling noise from 200,000 numbers. The shortcut matters because training a diffusion model needs x_t for random t thousands of times, and one jump is 1,000 times cheaper than 1,000 steps.

WHY CHOOSE THIS?

A closed form for any step, and a variance that never drifts: the process is easy to sample and easy to check.

WHY NOT THIS? (TRADEOFFS)

It only says how data is destroyed; all the difficulty is in reversing it, which is Day 140.

2. The week's model: a conditional DCGAN on all 60,000 images, scored

DEFINITION — start here, no prior knowledge assumed

Day 137's conditional DCGAN, built from the exercise parts, trained on all 60,000 training images (Day 137 used 20,000) for 10 epochs, and scored after epochs 1, 2, 5 and 10 with Day 138's Scorer on 2,000 fixed noise vectors, 200 asked for each digit. Whether an image shows the digit asked for is decided by Day 138's feature network.

REAL-WORLD SCENARIO

A final exam that uses every chapter of the course at once, marked with the grading scheme from the last chapter.

SPECIFICATION

Players	generator 74 → 64 × 7 × 7 → 32 × 14 × 14 → 1 × 28 × 28; discriminator 11 channels in
Training	60,000 images, batches of 128, 10 epochs, AdamGAN (lr 0.0002, b1 0.5), seed 0
Scored	IS, FID, precision, recall, nearest training image (Day 138), share drawn as asked
Real baseline	2,000 test digits in random order (Day 138: the stored order is not random)

IMPLEMENTATION

```
def train_week(X, y, sc, epochs, seed, at, k=64, batch=128):
    G, D = cgan_players(seed, k)
    oG, oD = AdamGAN(G.net, 2e-4, 0.5), AdamGAN(D.net, 2e-4, 0.5)
    rng = np.random.RandomState(seed); Zfix = np.random.RandomState(9).standard_normal((2000,
        k)).astype(np.float32)
    yfix = np.tile(np.arange(10), 200); log, t0 = [], time.perf_counter()
    for ep in range(1, epochs + 1):
        o = rng.permutation(len(X))
        for s in range(0, len(X) - batch + 1, batch):
            b = o[s:s + batch]; cgan_step(G, D, oG, oD, X[b], y[b], rng, k)
        if ep in at:
            F = net_generate(G, g_input(Zfix, yfix)); log.append((ep, sc(F), asked(sc, F, yfix),
                time.perf_counter() - t0))
    return G, log, F, yfix
```

ACTUAL OUTPUT

```
feature network (Day 138's part, 3 epochs, 63 s): test accuracy 0.9829
real test digits (2,000): IS 9.407, FID 2.996, precision 0.855, recall 0.884, nearest 4.49, named correctly
    0.990
generator 74 -> 64 x 7 x 7 -> 32 x 14 x 14 -> 1 x 28 x 28; discriminator 11 channels in; AdamGAN (lr 0.0002,
    b1 0.5);
60,000 images, batches of 128, 10 epochs (468 rounds each); scored on 2,000 fixed noise vectors, 200 asked for
    each digit
  epoch      IS      FID  precision  recall  nearest  asked digit drawn
      1   1.442  335.593      0.999   0.000     5.59              0.117
      2   4.659   96.525      0.415   0.099     6.27              0.143
      5   5.903   40.535      0.328   0.567     6.48              0.491
     10   7.580   20.200      0.442   0.573     6.35              0.834
(6387 s)
per digit at the end, 200 images each (FID with only 200 images is biased upwards, so real test digits are
    shown too):
   digit              0      1      2      3      4      5      6      7      8      9
   asked, drawn    0.88   0.92   0.89   0.81   0.91   0.74   0.83   0.89   0.75   0.72
   FID, GAN        69.5   66.3   61.1   59.9   45.2  117.7   69.7   52.7   52.2   50.5
   FID, real        6.0    4.8   22.8   13.9    6.7   14.8   13.7   10.3   11.9    7.9
Actual output from this lesson's run.
```

What it means

After epoch 1 the generator is in the collapsed phase Day 138 saw in every GAN: IS 1.442, precision 0.999, recall 0.000. It gets out by epoch 2 and improves steadily: FID 96.525, 40.535, 20.200 at epochs 2, 5 and 10, with recall 0.573 and the asked-for digit drawn 0.834 of the time. Real test digits score FID 2.996 and recall 0.884, so the gap is still large. Training took 6387 s.

Day 138's unconditional DCGAN scored FID 51.747 after 20 epochs on 20,000 images. This run differs in three ways at once (the label, three times the data, and the number of rounds), so the lower FID cannot be credited to any one of them.

Per digit, 5 is the weakest: drawn as asked 0.74 of the time with a per-digit FID of 117.7, against 14.8 for real fives. Per-digit FIDs use only 200 images, so they are biased upwards (real 2s score 22.8); only the comparison with the real row is meaningful.

WHY CHOOSE THIS?

Every part of the week works together, and the scores show where it still falls short (recall, the digit 5).

WHY NOT THIS? (TRADEOFFS)

Over 1.7 hours of training on a CPU for one seed; a second seed was not run, so run-to-run spread is unknown.

3. Against Day 133's Gaussian class models

DEFINITION — start here, no prior knowledge assumed

The simplest generative model of the week: for each digit, a mean image and a covariance matrix fitted to its training images, sampled and scored exactly like the GAN. Three versions: one covariance shared by all digits, or one per digit (full) with two amounts of regularisation added to the diagonal.

REAL-WORLD SCENARIO

Comparing a hand-tuned racing car with a sensible family car on the same track: the simple one may win on some measures.

IMPLEMENTATION

```
for kind, reg in (("shared", 1e-3), ("full", 1e-3), ("full", 1e-2)):
    t0 = time.perf_counter(); m = fit_gaussians(X[:60000].astype(np.float64), y[:60000], 10, kind,
        reg); rng = np.random.RandomState(0)
    F = np.empty((2000, 784), np.float32)
    for d in range(10):
        F[labels == d] = sample_class(m, d, 200, rng)
    F = np.clip(F, 0, 1); s = sc(F); a = asked(sc, F, labels); out[(kind, reg)] = (s, a, F)
```

ACTUAL OUTPUT

```
shared covariance, reg 0.001 (2 s): IS 4.449, FID 99.448, precision 0.794, recall 0.111, nearest 6.11, asked
    digit drawn 0.619
  full covariance, reg 0.001 (2 s): IS 8.486, FID 41.405, precision 0.844, recall 0.342, nearest 5.37, asked
      digit drawn 0.966
  full covariance, reg 0.01 (2 s): IS 8.207, FID 51.468, precision 0.863, recall 0.292, nearest 5.72, asked
      digit drawn 0.960
Actual output from this lesson's run.
```

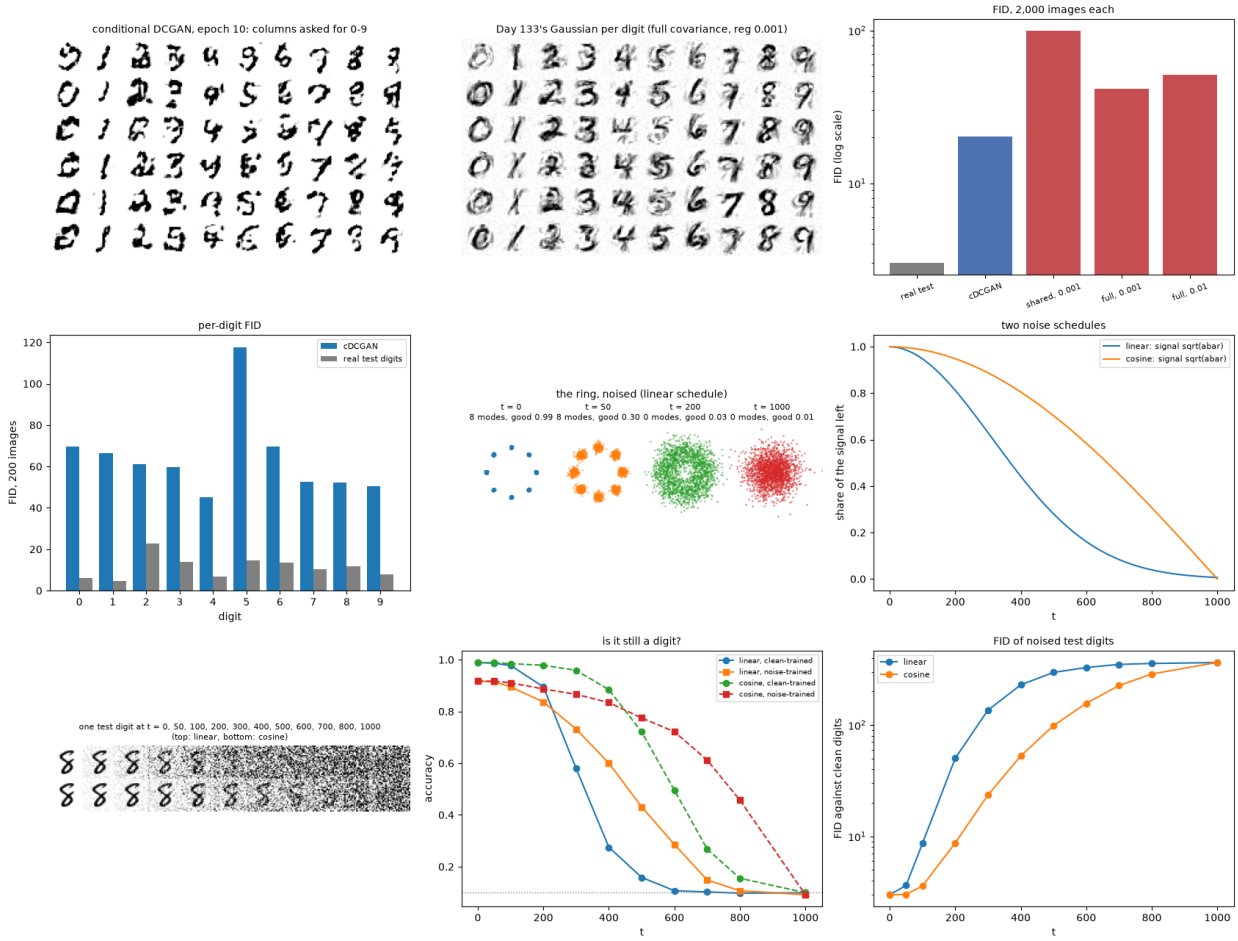
The week's models side by side (2,000 images each, from the run output above):

Model	FID	IS	Precision	Recall	Drawn as asked	Time
Real test digits	2.996	9.407	0.855	0.884	0.990	–
Conditional DCGAN, 10 epochs	20.200	7.580	0.442	0.573	0.834	6387 s
Gaussian, full, reg 0.001	41.405	8.486	0.844	0.342	0.966	2 s
Gaussian, full, reg 0.01	51.468	8.207	0.863	0.292	0.960	2 s
Gaussian, shared, reg 0.001	99.448	4.449	0.794	0.111	0.619	2 s

What it means

The GAN wins on FID (20.200 against 41.405) and recall (0.573 against 0.342): its digits cover more of the real variety. The best Gaussian wins on precision (0.844 against 0.442) and on drawing the digit asked for (0.966 against 0.834), in 2 seconds instead of 6387 s. A Gaussian sample is the digit's average shape plus correlated variation, so it is almost always the right digit, but grainy (top-middle panel of the figure) and less varied. That precision is high for grainy images fits Day 138's finding that precision can rise with noise.

A shared covariance is much worse (FID 99.448, drawn as asked 0.619): each digit varies in its own way. More regularisation (0.01) lowered recall (0.292) and raised FID (51.468).



Top: the GAN's samples, the Gaussian's samples, FID of each model. Middle: per-digit FID; the ring noised; the two schedules' remaining signal. Bottom: one test digit noised (linear, cosine); accuracy on noised digits; FID of noised digits.

WHY CHOOSE THIS?

A baseline that takes seconds tells whether a GAN's hours of training bought anything, and on which measures.

WHY NOT THIS? (TRADEOFFS)

No single score ranks the two: FID and recall favour the GAN, precision and the label favour the Gaussian.

4. The forward process on eight known modes

DEFINITION — start here, no prior knowledge assumed

Day 135's ring of eight Gaussians, a 2-D dataset where every mode is known, is pushed through the forward process. At each t, Day 135's mode_stats counts how many of the eight modes are still found and the share of points within 0.15 of a centre, and Day 138's Frechet distance compares x_t's fitted Gaussian with N(0, I), the point the process ends at.

REAL-WORLD SCENARIO

Ink drops in a glass of water: first eight sharp spots, then eight blurry clouds, then evenly tinted water.

IMPLEMENTATION

```
for t in (0, 1, 5, 10, 25, 50, 100, 200, 400, 700, 1000):
    xt = q_sample(x0, t, abar, rng); m, g, _ = mode_stats(xt); fd = frechet(*gauss(xt),
        np.zeros(2), np.eye(2))
    rows.append((t, m, g, fd, xt[:2000])); print(f"{t:6d} {np.sqrt(abar[t]):11.4f} {np.sqrt(1 -
        abar[t]):9.4f} {m:6d} {g:11.3f} {fd:22.4f}")
```

ACTUAL OUTPUT

```
Day 135's mode_stats on x_t (a sample is good within 0.15 of a centre); the Frechet distance (Day 138) between
x_t's fitted Gaussian and N(0, I), the end point of the process
     t  sqrt(abar)  noise sd  modes  good share  distance from N(0, I)
     0      1.0000    0.0000      8       0.988                0.3448
     1      0.9999    0.0100      8       0.986                0.3448
     5      0.9997    0.0264      8       0.971                0.3444
    10      0.9991    0.0435      8       0.925                0.3440
    25      0.9958    0.0919      8       0.643                0.3394
    50      0.9854    0.1702      8       0.298                0.3292
   100      0.9471    0.3209      8       0.097                0.2841
   200      0.8118    0.5839      0       0.028                0.1593
   400      0.4418    0.8971      0       0.015                0.0167
   700      0.0835    0.9965      0       0.013                0.0002
  1000      0.0064    1.0000      0       0.012                0.0001
for comparison, 20,000 exact N(0, I) samples: distance 0.0002
Actual output from this lesson's run.
```

What it means

The modes blur long before they vanish. Points stop being within 0.15 of a centre early: 0.925 at t = 10, 0.298 at t = 50, when the added noise has a standard deviation of only 0.1702. The eight clusters are still found at t = 100 and none by t = 200, by Day 135's mode count. The distance to N(0, I) falls from 0.3448 to 0.0167 at t = 400 and 0.0002 at t = 700, the same as for exact N(0, I) samples: from there on the data is pure noise as far as this measure can tell. A likely consequence for Day 140, not tested here: the reverse process must create the modes somewhere between t = 200 and t = 50.

WHY CHOOSE THIS?

A dataset with known answers shows exactly when the structure disappears.

WHY NOT THIS? (TRADEOFFS)

Two dimensions: what blurs early or late in 784-pixel images may differ, which section 5 checks.

5. The forward process on MNIST: linear against cosine schedule

DEFINITION — start here, no prior knowledge assumed

Test digits, scaled to [–1, 1], are noised to each t with two schedules: Ho et al.'s linear β, and Nichol and Dhariwal's cosine schedule, which defines A_t directly and removes the signal more evenly. Three measures: the feature network's accuracy (trained on clean digits), the accuracy of a softmax regression trained on digits noised to the same t (an estimate of how much digit information is left), and FID against clean digits.

REAL-WORLD SCENARIO

Turning down a radio's volume while the static rises: the question is when the song can no longer be recognised.

MATHEMATICS

$$\text{linear: } \beta_t = 0.0001 \ldots 0.02; \quad \text{cosine: } A_t = \cos^2((t/T + 0.008)/1.008 \cdot \pi/2) \,/\, \cos^2(0.008/1.008 \cdot \pi/2)$$

IMPLEMENTATION

```
for t in (0, 50, 100, 200, 300, 400, 500, 600, 700, 800, 1000):
    xt = q_sample(x0, t, abar, rng); img = ((xt + 1) / 2).astype(np.float32)
    ca = asked(sc, img, yt)[0]
    W, b = d116.fit_softmax(q_sample(xtr, t, abar, rng), ytr, 10, lam=1e-3, steps=200, lr=0.05)
    na = float(np.mean(d116.softmax(xt @ W + b).argmax(1) == yt)); s = sc(img)
    rows.append((t, np.sqrt(abar[t]), ca, na, s[1], img[:1].copy())); print(f"{t:8d}
        {np.sqrt(abar[t]):11.4f} {ca:14.3f} {na:14.3f} {s[1]:9.2f}")
```

ACTUAL OUTPUT

```
pixels scaled to [-1, 1] (variance 1 would be ideal; here it is 0.386); each x_t is mapped back by (x_t + 1) /
    2 for the feature network
columns: signal left sqrt(abar); the feature network's accuracy (trained on clean digits); the accuracy of Day
    116's softmax
regression trained on 10,000 digits noised to the same t (an estimate of what is left to find); FID against
    clean digits
linear schedule:
       t  sqrt(abar)  clean-trained  noise-trained       FID
       0      1.0000          0.990          0.917      3.00
      50      0.9854          0.986          0.914      3.63
     100      0.9471          0.978          0.895      8.65
     200      0.8118          0.895          0.837     50.28
     300      0.6296          0.580          0.732    135.81
     400      0.4418          0.275          0.600    229.01
     500      0.2803          0.159          0.431    295.96
     600      0.1609          0.107          0.286    327.38
     700      0.0835          0.102          0.148    349.44
     800      0.0391          0.098          0.107    355.96
    1000      0.0064          0.099          0.091    363.06
   the noise-trained accuracy first falls below 0.5 at t = 500
cosine schedule:
       t  sqrt(abar)  clean-trained  noise-trained       FID
       0      1.0000          0.990          0.917      3.00
      50      0.9960          0.988          0.917      3.02
     100      0.9859          0.985          0.910      3.58
     200      0.9480          0.979          0.887      8.74
     300      0.8871          0.959          0.866     23.54
     400      0.8047          0.884          0.835     52.74
     500      0.7027          0.721          0.775     98.76
     600      0.5838          0.495          0.722    156.34
     700      0.4507          0.268          0.612    225.69
     800      0.3067          0.155          0.457    286.38
    1000      0.0000          0.100          0.091    364.03
   the noise-trained accuracy first falls below 0.5 at t = 800
note: Day 116's default learning rate, 0.5, was unstable on these inputs in a first draft (linear schedule, t
    = 0, 50, 100,
200, 300, 500: 0.883 0.889 0.545 0.808 0.729 0.373, rising again after t = 100); 0.05 for 200 steps, 0.01 for
    500 and 0.05
for 500 all fell steadily and agreed within about 0.01, so 0.05 for 200 steps is used
Actual output from this lesson's run.
```

What it means

With the linear schedule the noise-trained accuracy falls below 0.5 at t = 500 (0.431); with the cosine schedule only at t = 800 (0.457). The linear schedule spends its last steps on what is already almost noise: from t = 700 on the signal left is under 0.0835 and the accuracy is near chance (0.148, 0.107, 0.091). The cosine schedule spreads the loss of information more evenly over the 1,000 steps, which is the reason Nichol and Dhariwal gave for it.

From t = 300 on (linear) the regression trained on noised digits beats the network trained on clean ones (0.732 against 0.580): a classifier that never saw noise underestimates what is left. Even so, the regression is a linear model (0.917 on clean digits, against the network's 0.990), so its accuracy is only a lower estimate of the information that remains.

The pixels have variance 0.386, not 1, so the variance of x_t is not constant here; the run says so and does not rescale. The regression's learning rate was tuned, and the run reports it: Day 116's default, 0.5, was unstable on these inputs (0.883 0.889 0.545 0.808 0.729 0.373), and 0.05 for 200 steps agreed with slower settings within about 0.01.

WHY CHOOSE THIS?

The cosine schedule keeps digits recognisable for longer, so fewer of the steps are wasted on pure noise.

WHY NOT THIS? (TRADEOFFS)

These are measures of the forward process only; whether the cosine schedule also generates better digits is not tested here.

Interview preparation — technical questions

Q: How do you sample x_t without running t steps?

A: $x_t = \sqrt{A_t}\, x_0 + \sqrt{(1 - A_t)}\, \varepsilon$, $A_t = \prod(1 - \beta_s)$; it matched 1,000 steps here (0.6259 against 0.6294).

Q: Why does each step shrink the signal by $\sqrt{(1 - \beta)}$?

A: So the variance stays 1: $(1 - \beta) + \beta = 1$. Without it the variance would grow with every step.

Q: How is a linear β schedule different from a cosine one?

A: Linear wastes the last steps on near-noise; cosine defines A_t as a squared cosine, removing signal evenly.

Q: How do you check that a build uses the parts unchanged?

A: Parse each exercise file with ast and check every function's exact text appears in the build (43 of 43).

Q: How do you condition a DCGAN on a label?

A: A one-hot vector joined to the noise, and ten label planes added to the discriminator's input (Day 137).

Interview preparation — theoretical questions

Q: Why compare a GAN with Gaussian class models?

A: A cheap baseline: here it lost on FID (41.405 against 20.200) but won on precision and the label.

Q: What does the forward process end at?

A: N(0, I): on the ring the distance to it was 0.0002 by t = 700, the same as for exact samples.

Q: Why might precision favour the Gaussian samples?

A: They are almost always the right digit; Day 138 found precision can even rise with noise.

Q: When is a noised digit no longer a digit?

A: A regression trained on noised digits falls below 0.5 at t = 500 (linear) and t = 800 (cosine).

Q: Why train the probe on noised data?

A: A classifier trained on clean digits underestimates what is left: 0.580 against 0.732 at t = 300.

Key takeaway and what's next

KEY TAKEAWAY

The week's parts assemble into a conditional DCGAN that beats Gaussian class models on FID and recall but not on precision or on drawing the digit asked for, so the choice of score decides the winner. The forward diffusion process destroys data by a schedule that can be written down exactly and sampled at any step in one line; the linear schedule wastes its last few hundred steps on what is already noise, the cosine schedule does not.

- 43 of 43 parts verbatim; one jump matches 1,000 steps (0.6259 against 0.6294 at t = 300).
- Conditional DCGAN, 10 epochs on 60,000 images: FID 20.200, recall 0.573, drawn as asked 0.834.
- Best Gaussian class model: FID 41.405, precision 0.844, drawn as asked 0.966, in 2 s.

- Ring: points leave their modes by t = 50 (0.298); indistinguishable from N(0, I) by t = 700.
- MNIST: the noise-trained probe falls below 0.5 at t = 500 (linear) and t = 800 (cosine).

Suggested watch order, if starting from zero

- 1. The diffusion model (DDPM) – the basics | Generative AI (TileStats) — the forward and reverse processes of a DDPM, from the basics.
- 2. What are Diffusion Models? (Ari Seff) — a visual explanation of how diffusion models work.

Day 140 preview

- **Week 21 begins: reverse diffusion.** Starting from pure noise, remove a little noise at a time: first with the exact best denoiser, which can be written down for the ring, then with one learned from data.